34 NP 完備性

Part VII Selected Topics

本章定位

NP完備性以位元編碼、量詞及可驗證證書界定難度；本章逐步建立核心歸約的構造、雙向語義與多項式成本，讓困難性證明可重建。

前置知識

決策問題、圖靈機、約化與布林電路。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 定義 P、NP、NP-hard、NP-complete。
- 建立 polynomial-time many-one reduction。
- 用 SAT、clique、vertex cover、Hamiltonian cycle 做證明。
- 34.1 Polynomial time
- 34.2 Polynomial-time verification
- 34.3 NP-completeness and reducibility
- 34.4 NP-completeness proofs
- 34.5 NP-complete problems

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.1 Polynomial time

決策問題、編碼與輸入長度

定義與前提

abstract problem 可把 instance 集合映到 answers。決策問題的答案只有 yes 與 no，因此可用語言 `L={x: answer(x)=yes}` 表示。

同一個數學物件可能有不同 encoding。例如整數 t 用 binary 表示約需 `lg(t+1)` 位，用 unary 表示需 t 個符號。多項式時間必須先指定 encoding，因為演算法的界限以實際輸入位元數衡量。

$$L \in \mathrm{P} \iff \exists\, \text{deterministic TM } M,\ \exists\, \text{polynomial } p : \forall x,\ M(x) \text{ halts in } p(|x|) \text{ and } (M(x) = 1 \iff x \in L).$$

`forall x` 不能寫成「存在某個 x」。P 的演算法必須對每個合格式處理，且錯格式也須在 polynomial time 拒絕或依規格處理。

P 的封閉性與 poly-time 組合

操作與推導

若程序 A 在 `p_A(n)` 時間產生可行 encoding，程序 B 在 `p_B(n)` 時間解第二個問題，先後執行成本為 `p_A(n)+p_B(p_A(n))`。多項式代入多項式仍為多項式。

這提供兩個常用結論。

1. 若 A 可在 polynomial time 直接求解，而 B 可把 instance 直接解算轉成 A，則 B 也可 polynomial-time 求解。
2. 多個 polynomial-time 轉換可串接，因為錯誤方向與長度爆炸都被 poly 界限控制。

但「演算法在某部真實電腦跑得快」不是成員證明。`O(n^100)` 仍屬 P，而常數很低的 exponential algorithm 可能在小 n 更實用。計算類別與工程效能是兩個問題。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

bit input、數值參數與 pseudopolynomial

可追蹤數例

令 $\mathrm{bitlen}(t)$ 為 t 的 binary 表示位數。若演算法成本 $O(nt)$，而 $t < 2^{\mathrm{bitlen}(t)}$，成本相對於輸入位元數可能是指數。

考慮 subset-sum：輸入 `n` 個整數與目標 B，DP 表有 `n(B+1)` 格，因此 `O(nB)`。B binary 編碼時，`nB` 不是輸入長度的多項式；這種以數值上界為參數、而非以位元長度為參數的演算法稱為 pseudopolynomial algorithm。

可追蹤族。 令 $n = m$、$t = B = 2^m$，並令 $a_i = 2^{m-1} + i$、$i = 1, \ldots, m$、$m \geq 2$，使每個 a_i 都有 $\Theta(m)$ bits。目標有 $m + 1$ bits，總輸入長度 $N = \Theta(m^2)$，但 DP 成本為

$$\Theta(m2^m) = 2^{\Theta(m)} = 2^{\Theta(\sqrt{N})}.$$

若它是 input-bit polynomial，對某個 c 與常數 C 最終應有 $m2^m \leq C(m^2)^c$；然而 $m2^m / m^{2c} \to \infty$，矛盾。

pseudopolynomial 不表示演算法無用。它表示「在 B 小或 unary encoding 下 polynomial」，不能因此宣稱 subset-sum 在 binary 標準編碼下屬於 P。

34.2 Polynomial-time verification

NP 的完整量詞

NP 類常被說成「答案可被快速檢查」。完整定義需要 certificate、長度上界與 deterministic verifier。

$$L \in \mathsf{NP} \iff \exists V,\ \exists\ \text{polynomials}\ p, q : \forall x,\quad x \in L \iff \exists y\ \big(|y| \le q(|x|) \wedge V(x, y) = 1\big),$$

且 V 是 deterministic algorithm，對所有 x、y 在 `p(|x|+|y|)` 時間內終止。y 稱為 certificate 或 witness。

量詞順序不能交換。對 yes instance，存在 witness；對 no instance，任意 witness 都不能讓 verifier 正確接受。所謂「猜 witness」的 NTM 說法，是把這個存在量詞轉成 nondeterministic computation path。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

verifier 的 soundness 與 completeness

何時失效

針對某個關係 `R(x,y)`，可定義 verifier 的兩種方向：

- completeness：若 `x in L`，存在 polynomial-length y 使 `R(x,y)=1`。
- soundness：若存在 y 使 `R(x,y)=1`，則 `x in L`。

合併兩者才是 `x in L iff exists y R(x,y)`。只有 completeness 可能接受錯誤 instance；只有 soundness 可能漏掉某些 yes instance。

以 Hamiltonian cycle 為例，certificate 是 n 個頂點的一條序列。Verifier 必須檢查：序列是所有頂點的一次排列、每個相鄰 pair 都是原圖邊、末點連回首點。四項全部成立才接受。一個只檢查「共有 n 個頂點」的 verifier 不 sound。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

P 包含於 NP 的證明

假設 `L in P`，存在 deterministic polynomial-time M 判定 L。定義 `V(x,y)` 忽略 y，直接執行 M(x) 並輸出 M 的答案。取空字串 `epsilon` 為 certificate，長度 0。

若 `x in L`，M(x)=1，因此存在 short witness epsilon 使 V=1。若 x notin L，因 M 對所有 y 都回 0，不存在 witness 使 V 接受。這同時得到 soundness 與 completeness。

因此 `P subseteq NP`。是否有逆包含 `NP subseteq P` 仍是 open problem；教材不能把「尚未找到 polynomial algorithm」誤寫成「已證明不存在」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.3 NP-completeness and reducibility

many-one reduction 的完整定義

`A <=_p B` 表示存在函數 f，滿足：

1. f 對 A 的每個合格式 input 都可在 `poly(|x|)` 時間計算。
2. f 的輸出是 B 的合格式。
3. 對所有 `x`，`x in A iff f(x) in B`。

方向是「把 source A 的 instance 轉成 target B 的 instance」。若想證明新問題 B 困難，可以把已知困難 A 轉成 B；不能把 B 轉成已知容易問題就下結論。

此 reduction 是 many-one，因為每個 x 只產生一個 B instance。它不允許根據求解器答案再回頭修改 x，也不允許呼叫 B solver 多次，除非那些呼叫本身可被包進合法 polynomial-time 轉換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向語義：completeness 與 soundness

`iff` 要拆成兩個檢查：

- completeness direction：`x in A implies f(x) in B`。A 的每個 yes instance 都要構造出 B 的 yes instance。
- soundness direction：`f(x) in B implies x in A`。任何由 f 產生的 B yes instance 必須能解回 A 的 yes instance。

名稱在文獻中可能因視角互換；解題時應直接寫方向，避免只寫「sound/complete」造成歧義。

以 CLIQUE 為例，轉成的圖若存在 k-clique，具體 clique 必須提供足以還原一組 satisfying assignment 的資訊。若圖的 size、k 或邊的 gadget 允許不存在的 assignment，soundness 就壞掉。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

transitivity 定理與完整證明

定理。 若 `A <=_p B` 且 `B <=_p C`，則 `A <=_p C`。

證明。 令 f 是 A 到 B 的 reduction，g 是 B 到 C 的 reduction。定義 `h(x)=g(f(x))`。

若 `x in A`，由 f 的 iff 得 `f(x) in B`；再由 g 的 iff 得 `g(f(x)) in C`。反向若 `g(f(x)) in C`，由 g 得 `f(x) in B`，再由 f 得 `x in A`。因此 `x in A iff h(x) in C`。

令 f 在 `p_f(n)` 時間終止，g 在 `p_g(m)` 時間終止。因 `|f(x)| <= p_f(|x|)`，算 `h(x)` 的時間至多 `p_f(n)+p_g(p_f(n))`。多項式合成仍為多項式。故 A <=_p C。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-hard 與 NP-complete

語言 L 是 NP-hard，若對每個 `A in NP` 都有 `A <=_p L`。語言 L 是 NP-complete，若：

1. `L in NP`；
2. L 是 NP-hard。

若已知 C 是 NP-complete，且 `L in NP`，要證明 L 是 NP-complete，只需證 `C <=_p L`。任取 `A in NP`，因 C NP-hard 得 `A <=_p C`；再用已知 `C <=_p L` 與 transitivity 得 `A <=_p L`。

缺少 `L in NP` 只能得到 NP-hard，不能得到 NP-complete。缺少 `C <=_p L` 則沒有把已知困難性傳到 L。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第一個 NP-complete 問題的構造模型

CIRCUIT-SAT 輸入是有限、有向無環的 Boolean circuit，gate 為 fan-in 至多二的 AND、OR 與一輸入 NOT；問是否有一組輸入使指定輸出為 1。certificate 只需原始輸入 bits，依拓撲序計算每個 gate、檢查輸出，時間與 gate/wire 數成線性，故在 NP。

要證 NP-hard，任取 L in NP，固定其 deterministic verifier V、certificate 長度界 q(n) 與時間界。給 x，n=|x|，電路 C_x 的自由輸入必須是 witness y，而不是把一個固定 computation 的接受狀態當作未知。

可將witness規格化為固定長度q(n)+b(n)，其中長度欄b(n)=1+⌈$\log_2(q(n)+1)$⌉，至少1 bit。前b(n)位編碼實際長度ℓ，後q(n)位放witness並補零；新verifier先檢查$0 \le \ell \le q(n)$，再讀前ℓ個witness bits並模擬V。q(n)=0時仍有一位長度欄，只允許$\ell=0$。這不改變「存在可接受witness」，也不會把不同長度默認成相同字串。

Lin Ching https://github.com/xinhi6716/computer-science-self-study

由局部 transition 逐 gate 展開計算

固定一部模擬此verifier的單帶deterministic TM，讓它於接受或拒絕後停留在同一終止狀態。多帶轉單帶至多帶來多項式時間損失。令T(n)為包含witness長度後的多項式步數界，初始帶長N=O(n+q(n)+1)，包含前頁至少一位的長度欄，q(n)=0也有定義。

一步最多移動一格，所以只需表示 O(N+T) 個 tape cells，兩端額外保留 T+1 空白格，頭不會碰到外界。每格標籤包含 tape symbol，以及「此格是否有頭、若有則是哪個 state」。機器字母表與狀態集固定，故每格用固定 b bits。

時間 t+1 的第 j 格標籤，只取決於時間 t 的 j-1、j、j+1 三格標籤。這是固定 3b 輸入的 Boolean 函數。逐個輸出 bit，用真值表寫成若干 minterms 的 OR，每個 minterm 為輸入 bit 或 NOT bit 的 AND，再把多輸入 AND/OR 拆成二輸入 gates。因 b 與機器固定，每個局部更新只需 O(1) 個 gates。

逐層接線、規模與 iff 證明

初始層將 x、空白格、初始頭位置與 state 接成常數，只有規格化 witness bits 自由。對 t=0,...,T-1，每個 tape cell 複製上述固定局部電路，將其輸出接到下一層相同 cell。最後用 OR tree 檢查第 T 層是否有接受 state 標記。

總 gate 數為 O(T(N+T))，wire 數同階，gate 索引使用 O(log(T(N+T))) bits；列舉時間層、cell 與固定真值表即可在多項式時間輸出電路，沒有隱藏指數大小的硬體。

對每組 y，依 t 歸納：第零層是合法初始 configuration；局部 transition 產生恰好下一 configuration。因此所有層都是 V(x,y) 的真實執行，無效局部標籤永遠不會由合法初始層出現。接受狀態持續不變，故 C_x(y)=1 iff V(x,y) 接受。

若 x in L，存在接受 y，於是 C_x 可滿足；若 C_x 可滿足，其自由輸入 y 使 V 接受，於是 x in L。每個 NP 語言都如此多項式轉入 CIRCUIT-SAT，加上 NP membership，完成第一個 NP-complete 證明。

34.4 NP-completeness proofs

CIRCUIT-SAT 到 SAT 的局部 gate 約束

為每條 wire（可共享同一 gate output）設一變數，並把輸出 wire z_out 本身作為 unit clause，要求它為真。每個 gate 加入輸出等於 gate 函數的條件：

Gate 關係	等價 CNF clauses
z = u AND v	(¬z∨u) AND (¬z∨v) AND (z∨¬u∨¬v)
z = u OR v	(z∨¬u) AND (z∨¬v) AND (¬z∨u∨v)
z = NOT u	(z∨u) AND (¬z∨¬u)

例如 AND 的前兩 clauses 強迫 z=true 時 u,v 皆真；最後一 clause 強迫 u=v=true 時 z=true。OR 與 NOT 可同樣分真假兩向直接檢查。常數 wires 用 unit clause 固定。

電路接受輸入可延伸為所有 wire 的真值，滿足每個局部 clause 與 output clause。反向滿足公式時，局部等價式沿拓撲序迫使每個 wire 等於實際計算值，output=true，所以原電路可滿足。每 gate 至多三 clauses，總長 O(gates+wires)，不把共享子電路反覆展開成指數長樹狀公式。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

SAT 到 3-CNF 的 parse-tree gadget

任意公式先解析成 NOT、一輸入或 AND/OR 二輸入樹；例如 implication 可用 $\neg a \vee b$ 改寫，其他固定 arity connective 亦用固定大小 truth-table gadget。為每個內節點設新變數，使用前頁 gate clauses，根變數設真。若同一節點輸入重複，先在所得 clauses 去除重複 literal；含 l 與 $\neg l$ 的 clause 是 tautology，可整個刪掉。

此時 clauses 長度至多三。若要求每 clause 恰含三個相異變數，逐 clause 使用新的 padding variables：

- 兩 literal $(a \vee b)$ 改為 $(a \vee b \vee p) \wedge (a \vee b \vee \neg p)$。$a \vee b$ 真則任意 p 都可；若假，兩式分別強迫 p 與 $\neg p$，矛盾。
- 一 literal a 改為四式 $(a \vee p \vee q)$、$(a \vee p \vee \neg q)$、$(a \vee \neg p \vee q)$、$(a \vee \neg p \vee \neg q)$ 的 AND。a 真則皆真；a 假時，任何 p,q 指派都令其中一 clause 為假。
- 空 clause 表示不滿足，直接輸出上述 padding 過的 $z \wedge \neg z$；空 AND 表示恆真，輸出固定可滿足的三變數 clause。

原 assignment 可用每個 parse node 的真值延伸；反向沿樹恢復所有節點語義。padding 保留每 clause 的真假要求，因此整體只保留 equisatisfiability，而非把加入新變數後的公式當作原變數的逐字同一函數。每節點常數 clauses，每 clause 至多擴成四個，所以大小與建構時間皆多項式。

長 clause 拆分與可追蹤 gate 例

若已得到長 clause $l_1 \vee \cdots \vee l_r$，$r > 3$，也可用 chain：

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \\ \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

若所有 l_i 都假，首式逼 $y_1 = \text{true}$，中間逐次逼下一個 $y_j = \text{true}$，末式卻要求最後 $y_{r-3} = \text{false}$，矛盾。若至少一個 l_i 真，令 y_j 表示前 $j+1$ 個 literals 是否全假，即可逐式滿足。此拆分用 $r-3$ 新變數、$r-2$ clauses；不能直接用 distributivity 把任意公式轉 CNF，那可能產生指數長度。

例 F=(a∨b) AND ¬a，設 u=a∨b、v=¬a、z=u AND v，給三組 gate clauses 並加 z。a=false,b=true 時 u=v=z=true，滿足所有約束。若 a=true，NOT gadget 強制 v=false，而 z 與 AND gadget要求 v=true，故這類指派不能假造接受。這同時示範 witness 延伸與反解。

原書 3-CNF 定義為三個相異 literals；本教材的主要 reduction 進一步可規格化為三個相異變數。出現重複 literal 的輸入仍可先依上述去重、padding 規則處理，不影響後續 CLIQUE occurrence 構造。

3-CNF-SAT <=_p CLIQUE 的構造

令 F 為 3-CNF formula，記 `|F|` 為其實際 binary encoding 長度。若 F 有 k 個 clauses，為每個 clause 中的每個 literal occurrence 各建一個 vertex，並令 CLIQUE 的參數也為 k。非退化情形取 `k>=1`；空 AND formula 沒有 clauses，恆為真，另行映射到 `(空圖,0)`。

邊的規則正好兩條：

1. 同一 clause 內的 vertices 彼此不相連。
2. 不同 clauses 的兩個 vertices 相連，若且唯若它們不是互補 literals。

每個 clause 恰有三個 occurrence，因此 vertices 數是 $3k$。先把實際出現的變數名稱編為n個ID（$n \le 3k$），逐一檢查vertex pairs需$O(k^2)$次$O(1)$ word比較。若逐bit計費，ID比較及edge-list端點各需$O(\log(n+k))$ bits，建圖可用$O(|F| \log|F| + k^2 \log(n+k))$的多項式bit上界；若輸出adjacency matrix則有$\Theta(k^2)$個bits。不能把任意長變數名稱的字串比較無條件當作$O(1)$。

可追蹤 3-CNF-SAT 例子

令

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

取 assignment `x1=false, x2=false, x3=true`。三個 clauses 都為 true。

選 literal occurrences：第一 clause 的 `x3`、第二 clause 的 `¬x1`、第三 clause 的 `x3`。它們來自不同 clause，因此建立跨群 edges；彼此不是 complement，所以三點兩兩相連，形成 size 3 clique。

這個 assignment 並不是唯一 witness。若改挑第一 clause 的 x1 與第二 clause 的 ¬x1，這兩個 occurrence 會因互補而缺邊，不能一起組 clique；在目前 assignment 下 x1 也為假，不應被選作真 literal。這正好說明 graph 不會把矛盾 literals 同時當成一致 witness。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

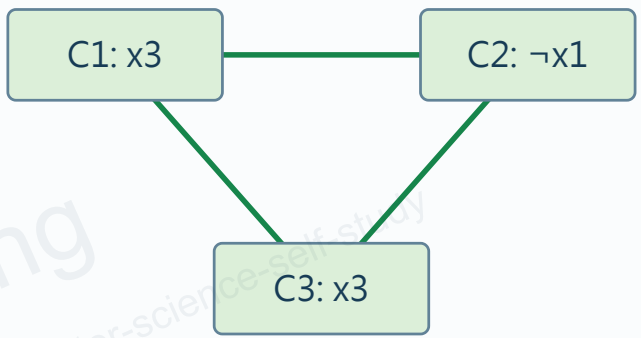
可追蹤 3-CNF-SAT 例子 (續)

不同 clause 且非互補，才連邊

C1 / C2	¬x1	x2	¬x3
x1	×	✓	✓
¬x2	✓	×	✓
x3	✓	✓	×

✓ 相連；× 互補而缺邊

選到的三個 occurrence 彼此一致



x1=0，x2=0，x3=1：每群一個真 literal

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3-CNF-SAT 到 CLIQUE 的 iff 證明

Completeness。 若 F satisfiable，固定一組 satisfying assignment。每個 clause 至少有一個 true literal occurrence，選其中一個。因 assignment 一致，兩個被選 occurrences 不會互為 complement，所以對應 vertices 之間都有邊。k 個 clauses 得到 k-clique。

Soundness。 若 G_F 有 k-clique，同一 clause 的 vertices 沒有邊，所以 clique 必須在每個 clause 中恰好選一個 occurrence。任意兩個被選 literals 都有邊，代表它們不互為 complement。因此同一個 variable 不會同時被要求為 true 與 false。令每個被選 variable 取得使其 literal 為真的值；未選 variables 任意指定；這些要求彼此一致，且每 clause 有一個 true literal，故 F satisfiable。

兩方向合併即 `F satisfiable iff G_F has k-clique`。CLIQUE 也有 polynomial certificate（k 個 vertices）與 checker，所以 CLIQUE 是 NP-complete。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CLIQUE、INDEPENDENT-SET 與 VERTEX-COVER

independent set S 是圖中任兩點都沒有 edge 的集合。vertex cover C 是每條 edge 至少一端在 C 的集合。

對 n 點、無 self-loop 的無向圖 G=(V,E)，令 `Gbar=(V, {uv: u!=v and uv notin E})`。同一 pair 在 G 有 edge 若且唯若在 Gbar 沒有 edge。因此：

$$C \text{ is a clique in } G \iff C \text{ is an independent set in } \overline{G}.$$

若 S 是 independent set，則 V\S 是 vertex cover。假設有 Gbar 的 edge 未被 V\S 覆蓋，它的兩端都在 S，違反 S independent。反向若 V\S 是 vertex cover，任何 Gbar edge 不可能兩端都在 S；所以 S independent，也是在 G 的 clique。

令 `k in {0,...,n}`。把上面兩步串起，得到

`CLIQUE(G,k)` iff `VERTEX-COVER(Gbar,n-k)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study/

CLIQUE、INDEPENDENT-SET 與 VERTEX-COVER （續）

`(G,k) -> (Gbar,n-k)` 與反方向的 `(H,l) -> (Hbar,n-l)` 都以 `O(n^2)` 次配對檢查建立補圖。這裡CLIQUE可說「至少k點」或「恰k點」，因較大clique任取k點仍是clique；VC採「至多n−k點」，較小cover可補點至恰n−k點，所以同一iff成立。這需0≤k≤n；範圍外門檻先直接判定並映射至固定yes/no實例，不能產生負大小certificate。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CLIQUE 與 VERTEX-COVER 的可追蹤例

令 G 有頂點 {1,2,3}，edges {12,23}，取 k=2。{1,2} 在 G 是 clique。

補圖 Gbar 只有 edge {13}。{1,2} 在 Gbar 是 independent set，因為 1 與 2 沒有 edge。其補集 {3} 是 Gbar 的 vertex cover，大小為 3-2=1，且它覆蓋唯一 edge 13。

轉換是 VC(Gbar, n-k)。所以 CLIQUE(G,k) iff VERTEX-COVER(Gbar,n-k)。這個等價直接給出雙向約化：complement 本身就是 polynomial-time、可逆的 encoding transformation。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.5 NP-complete problems

VERTEX-COVER 到 HAMILTONIAN-CYCLE 的端口 gadget

HC certificate 是全部頂點的排列，逐邊與閉合檢查給 NP membership。下面完整建立 NP-hardness，不需要平面 crossing gadget；圖中交叉畫線不代表交叉處是頂點。

對原圖每邊 {u,v} 建十二個新點 a1,...,a6、b1,...,b6，其中 ai=[u,v,i]、bi=[v,u,i]。內邊共十四條：

- 兩條鏈 a1-a2-a3-a4-a5-a6 與 b1-b2-b3-b4-b5-b6，共十條。
- 四條交叉連結 a1-b3、a3-b1、a4-b6、a6-b4。

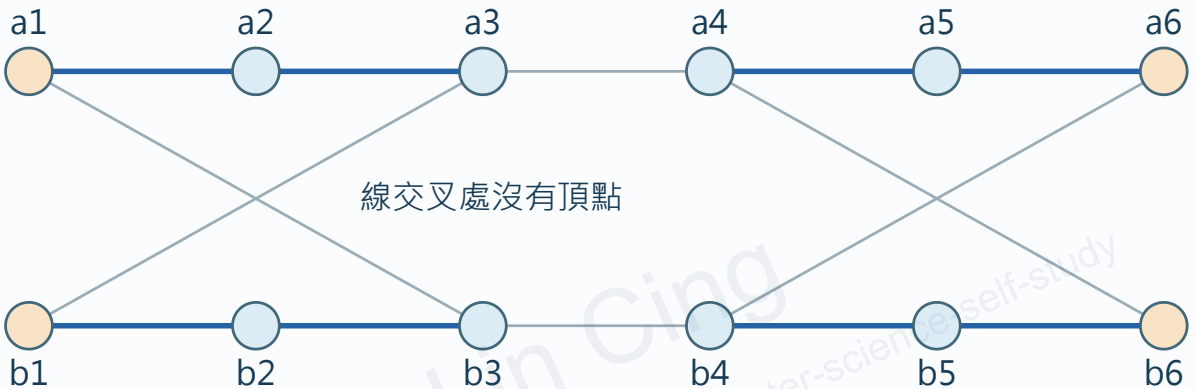
只有四個端口 a1,a6,b1,b6 可接 gadget 外部。其他八點在最後 HC 中必有兩條內邊。a2,b2,a5,b5 在 gadget 本身 degree=2，故它們的八條相鄰鏈邊被迫選入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.5 NP-complete problems （續）

HC gadget：只有 a1、a6、b1、b6 能接外部

a_i=[u,v,i]；b_i=[v,u,i]



藍色8邊由 a2、a5、b2、b5 的 degree=2 強迫選入；灰色6邊仍待決定。

兩條中邊 a3–a4、b3–b4 決定其餘4條cross edges；橘色為4個端口。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

gadget 只有三種合法的覆蓋方式

餘下選擇由中間兩邊 a3-a4、b3-b4 決定。每個內部點仍需 degree=2，故這兩邊一旦選或不選，四條交叉連結的使用亦被迫確定。

中邊選擇	對全部十二點的路徑覆蓋
兩條都選	a1-a2-a3-a4-a5-a6，以及 b1-b2-b3-b4-b5-b6，兩條 disjoint paths
只選 b3-b4	a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6，一條 a1 到 a6 的完整 path
只選 a3-a4	對稱的一條 b1 到 b6 的完整 path
兩條都不選	上下各形成一個封閉六點 cycle，與外部無法連成單一 HC，因此禁止

這四種窮盡所有可行內度配置，故合法 HC 只能使用前三種。尤其不能從 a 端口進來卻沿 b 的外部鏈離開；也不能讓十二點完全無人覆蓋。此局部事實正好表示原邊至少有一個端點被選。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中邊如何強迫端口的外部使用

令x表示是否選a3–a4，y表示是否選b3–b4，值為0或1。內點a3已有a2–a3，所以a3–b1必取1–x；a4同理使a4–b6取1–x。b3,b4則使a1–b3與a6–b4取1–y。

因此a1,a6的內部度都是2–y，還需要的外部度為y；b1,b6的外部度為x。端口不可能一邊連外、一邊封閉：同一原頂點側的兩端總是同時接外或同時不接外。

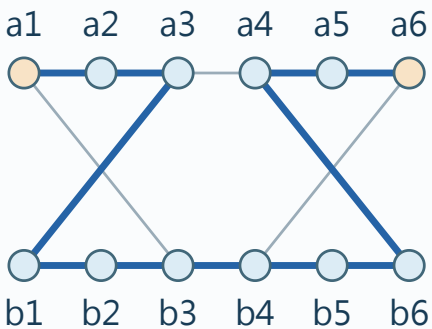
x=y=0時，下列兩個六點cycle已各自用滿度2：a1–a2–a3–b1–b2–b3–a1，以及a4–a5–a6–b4–b5–b6–a4。它們不能接到selector，故不可能是整張圖的單一HC。其餘三種配置的全部路徑如下圖，逐點可核對沒有漏點或換側離開。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中邊如何強迫端口的外部使用 （續）

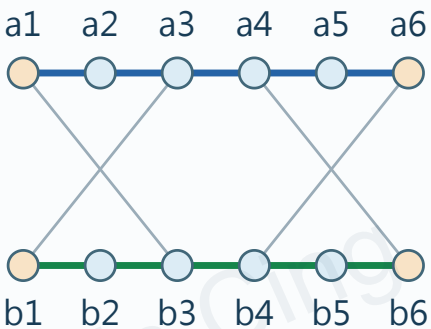
三種合法模式都不會跨原頂點身分離開

只走u側：12點



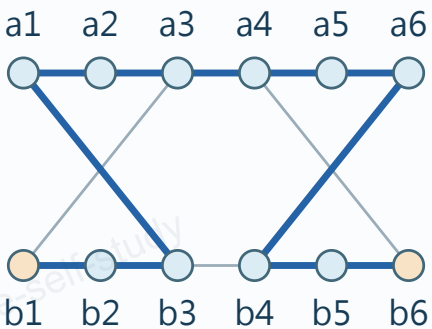
x=0，y=1

u、v各走6點



x=1，y=1

只走v側：12點



x=1，y=0

灰邊未選；藍 / 綠是各條路徑；橘色端口各需一條外邊補到 degree=2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

串接原頂點的 incident gadgets 與 selectors

VERTEX-COVER 輸入採「大小至多 k」。先移除孤點，不影響最小 cover。k<0 或 k=0 且有邊，輸出固定 no HC instance；無邊或 k 大於等於剩餘頂點數，輸出固定 yes triangle。以下可假設無孤點且 1<=k<|V|；一個較小 cover 可以補入其他非孤點成恰 k 點。

對每個原頂點 u，任排其鄰居 u(1),...,u(d)。把各 gadget 的 u 側依序串接：加入 [u,u(i),6]-[u,u(i+1),1]，i=1,...,d-1。這些連結使一條外部路徑一旦選 u，便沿它全部 incident gadgets 走到末端。

另建 k 個 selector s1,...,sk。每個 selector 都接到每個 u 鏈的首端 [u,u(1),1] 與末端 [u,u(d),6]，沒有其他外邊。圖共有 12|E|+k 點；十四內邊乘 |E|，串鏈至多 2|E|，selector 連邊 2k|V|，全為多項式規模。原圖有向無向不可混淆，此構造與目標皆為無向圖。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

VC 到 HC 的完整 iff

若 C={u1,...,uk} 是 cover，將每個被選 u 的 incident-gadget 鏈作一條 cover path。原邊若只有 u 被選，使用從 u 側進出的十二點完整 path；若兩端都選，使用兩條各六點 path。因每條原邊至少一端被選，每個 gadget 的十二點恰好各被訪問一次。依序用 s1、u1 鏈、s2、u2 鏈、...、sk、uk 鏈、s1 串成一個 cycle，亦各訪 selector 一次，故為 HC。

反向若新圖有 HC，移去 k selectors 後，cycle 被切成 k 條 disjoint cover paths。局部 gadget 的三模式禁止切換端點身分；每條 path 必沿同一原頂點 u 的 incident 鏈從頭到尾。不同 paths 不能選同一 u，否則會共用其鏈端或 gadget 點。取這 k 個 u 為 C。每個 gadget 的點都被 HC 訪到，而三種模式至少有 u 或 v 的 cover path 經過，所以每條原邊 {u,v} 至少一端在 C。得到大小 k 的 vertex cover。

因此 VC(G,k) iff HC(G')；構造、反解與規模皆多項式。固定 yes/no 例處理前處理的退化情況，不需用不存在的空鏈湊 selectors。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VC gadget 的小例

原圖為 path u-v-w、k=1，可選 cover {v}。建兩個十二點 gadgets W_uv、W_vw；把 v 側第一個 gadget 的端口 6 接第二個 gadget 的端口 1，再用唯一 selector 接整條 v 鏈的首尾。

兩條原邊只有端點 v 被選，所以每個 gadget 都由 v 側完整走過十二點。selector 與兩個 gadgets 共二十五點形成 HC。反之若選 u 的鏈，W_vw 沒有任何被選端點，無法覆蓋，因此不會假造 cover {u}。

完整二十五點cycle可由以下兩條十二點路徑接成。記U=W_uv、V=W_vw；U的b側代表v，V的a側代表v：

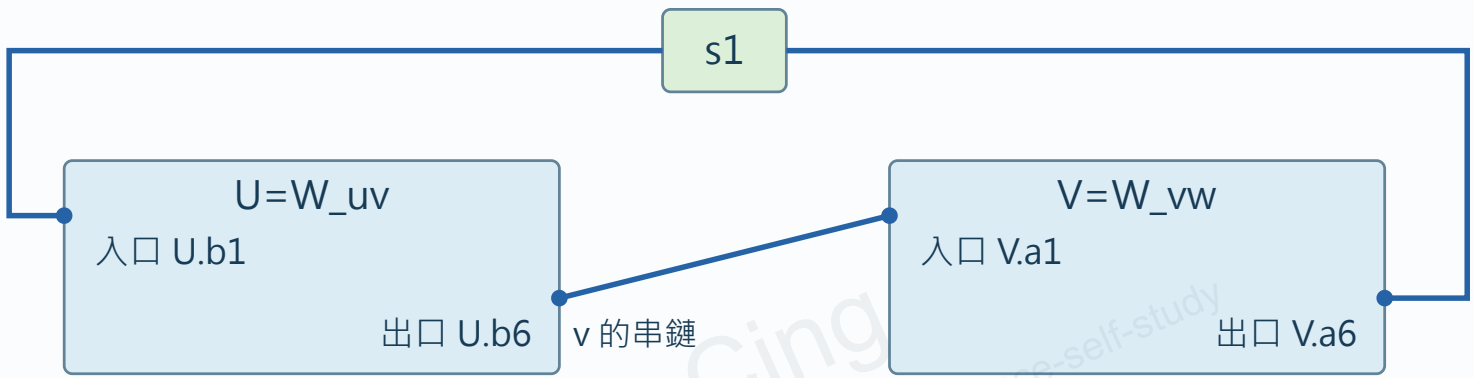
- U：b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6。
- V：a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6。

由s1→U.b1開始，走完U後經串接邊U.b6→V.a1，再走完V，最後V.a6→s1閉合。U與V的同名點是不同頂點，故24個gadget點加s1恰25點，每點只出現一次。每個相鄰pair都能在十四內邊、串鏈或selector邊中逐項查得。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

VC gadget 的小例 (續)

原圖 u−v−w、cover={v}：唯一 selector 串成25點 cycle



各框內是已列出的完整12點路徑，U與V的同名點互不相同。

此圖顯示選定的HC；完整外邊依selector與incident-chain構造定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

HAMILTONIAN cycle <=_p TSP decision

給 HC instance `G=(V,E)`，`n=|V|`。建立完全圖 `K_n`，邊權重：

`w(u,v)=1` 若 `{u,v} in E`，否則 `w(u,v)=2`。令門檻 `B=n`。

HC 到 tour。 G 的 Hamiltonian cycle 有 n 條原圖 edges，每條 weight 1，因此在 K_n 是 cost n 的 tour。

Tour 到 HC。 K_n 任一個 tour 恰有 n 條邊。若其 cost <= n，且每條 weight 至少 1，則每條邊都必須 weight 1。這些邊都在 G，tour 因而也是 G 的 Hamiltonian cycle。

構造是 `O(n^2)`。所以 `HC(G) iff TSP(K_n,w,n)`，且 TSP decision的 certificate 是一條 tour，檢查排列與總成本即可，故 TSP decision in NP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TSP 實例追蹤

令G是三角形，edges={12,23,31}，n=3。完全圖的三條邊全部權重1，因此tour 1→2→3→1的成本3，門檻B=3。

門檻 B=3。此 tour cost 恰好等於 B，對應 G 的 Hamiltonian cycle。

若原圖刪去31，構造出的完全圖仍有31，但權重改為2。任何三點tour都含三條邊，成本1+1+2=4>B，所以不存在合格tour，與原圖沒有HC一致。對n<3的簡單無向圖，本章HC無解，直接映射至固定no TSP instance；上述每tour恰n邊的主要論證採n≥3。

optimization 版 TSP 要求真正最小總成本；decision 版只問是否存在 cost <= B。若 optimization algorithm 在 polynomial time 回傳最佳值，決策版可直接比較 B；決策版 NP-hard 因而支持 optimization 版也難以精確 polynomial-time 求解。

優化問題與決策問題

很多 NP-complete 問題有 optimization counterpart。例：TSP 最佳 tour、SET-COVER 最少集合數、MAX-CLIQUE 最大 clique size。

若優化版在多項式時間可精確解，直接把最佳值與門檻比較便能解決策版。另一方向，若已有正確的決策 oracle，可對整數門檻binary search求最佳值。決策版NP-hard且精確優化可解時，便能解全部NP語言；故除非P=NP，這些精確優化問題不會有多項式時間演算法。

binary search 有兩個假設：

- 成本或答案介於可由 input 多項式位元表示的上下界；
- decision procedure 對所有門檻都正確。

得到最佳值不等於直接得到 witness。若也要輸出構造，需額外做 reconstruction，例如逐步固定決策並重跑 decision oracle；不能把「知道最佳值」冒充成「已回傳最佳解」。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

classification 的精確邊界

名稱	精確定義	已能推出什麼	不能直接推出什麼
NP-hard language L	每個 `A in NP` 都有 `A <=_p L`	若 `L in P`，則 `P=NP`	L 本身在 NP
NP-complete language L	`L in NP` 且 L NP-hard	它有一個 polynomial verifier，且可承接任一 NP source 的 reduction	一個 optimization 程式自動成為 language
CLIQUE decision	對 input `(G,k)` 回答是否存在 size-k clique	本節已證它在 NP 且 NP-hard	它輸出最大 clique
MAX-CLIQUE optimization	對 G 輸出最大 clique size（可再要求 witness）	若另有 polynomial algorithm，可由答案解 CLIQUE decision	optimization 的問題型別可直接套用 `NP-complete` 定義

`NP-hard` 是 reduction 的普遍性，`NP-complete` 額外要求 membership。decision 回答 yes/no；optimization 輸出最佳值，search 再輸出 witness。證明可互相解出時，要說明程式如何使用一個問題的完整 output，以及是否真的得到另一問題要求的答案。

SUBSET-SUM、bit encoding 與 DP

decision 版 SUBSET-SUM 問：給正整數集合 S 與 t，是否存在子集合元素和恰為 t。

Certificate 是一組 selected indices；verifier 檢查每個索引在範圍內、不重複，且所選整數和恰為 t。長度與檢查時間 polynomial，所以 SUBSET-SUM in NP。

DP 狀態 `dp[i,s]` 表示前 i 個 elements 可否達和 s。轉移為

`dp[i,s]=dp[i-1,s] or (a_i<=s and dp[i-1,s-a_i])`。

計算量 `O(n t)`。這是 pseudopolynomial，不是以 t 的 binary bit length 衡量的 polynomial。NP-hardness 的完整無進位數位表與兩向證明接在下頁；其巨大數值仍有多項式 bit 長，與此 DP 的 pseudopolynomial 界限一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3-CNF-SAT 到 SUBSET-SUM 的數位表

先把公式規格化為 n 個變數、k 個 clauses，每 clause 三個相異變數、無互補 tautology，每個變數至少出現一次；前面的去重/padding 處理可達成。恆真與恆假公式另輸出固定 yes/no 整數實例。

建立十進位 n+k 欄，前 n 欄對應變數，後 k 欄對應 clauses。這是緊湊建立整數的方式，最終仍以 binary 編碼交付。

- 每變數 xi 建兩數 T_i、F_i。兩者的第 i 個變數欄為 1，其他變數欄為 0。T_i 在包含正 literal xi 的 clause 欄為 1，F_i 在包含 ¬xi 的欄為 1，其餘為 0。
- 每 clause j 加 slack 數 S_j、S'_j，只在該 clause 欄分別放 1、2，其他欄為 0。
- target 的每個變數欄為 1，每個 clause 欄為 4。

總共 2n+2k 個正整數，每數 n+k 位十進位，故每數 binary 長度 O(n+k)，輸出總 bit 長 O((n+k)^2)，建構為多項式。若輸入要求集合而非 multiset，上述非退化假設保證兩個同變數數字不相同；不同變數的高位欄不同，slack 欄也不混同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

為何無進位，以及兩向解碼

任意子集合在每個變數欄至多加到 2；每個 clause 含三個 distinct literals，對應 T_i/F_i 數最多提供三個 1，slack 最多加 1+2=3，因此 clause 欄至多 6<10。任何子集合都不會進位，不只是正確 witness 才不進位。

若原式可滿足，每變數依真假選 T_i 或 F_i，使變數欄和全為 1。每 clause 的真 literals 數 r 為 1、2 或 3，可用 slack 子集合湊 4-r：r=1 選 1+2，r=2 選 2，r=3 選 1。所有欄符合 target，得到 subset-sum yes。

反向若數字子集合和為 target，無進位迫使每變數欄恰為 1，所以 T_i、F_i 恰選一個，定義一致 assignment。clause target 為 4，但兩 slack 合計至多 3，所以至少一個被選 T_i/F_i 在該欄有 1，對應原 clause 中一個真 literal。每 clause 都滿足。兩向與多項式規模一起完成 NP-hardness，再加索引 certificate 的 NP membership，SUBSET-SUM 為 NP-complete。

可手算的無進位例子

取 F=(x1∨x2∨x3) AND (¬x1∨x2∨¬x3)，欄序 x1,x2,x3,C1,C2：

數	十進位欄表示
T1,F1	10010，10001
T2,F2	01011，01000
T3,F3	00110，00101
S1,S1'	00010，00020
S2,S2'	00001，00002
target	11144

assignment x1=true,x2=true,x3=false 選 T1+T2+F3=11122。每個 clause 真 literal 數都是二，再選 S1'=20、S2'=2，得 11144。沒有進位；高位每欄的 1 也直接證明每個變數只選一個真值數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

可手算的無進位例子 （續）

數值可能與 10^{n+k} 同階，但 binary 表示僅 $O(n+k)$ bits。這正是 reduction 可多項式輸出，而 $O(nt)$ 精確 DP 仍非輸入 bit 長度多項式的原因。

T1+T2+F3=11122；再加 slack 00022，得到 target 11144

	x1	x2	x3	C1	C2	
T1	1	0	0	1	0	變數欄各恰為1
T2	0	1	0	1	1	兩個 clause 各有
F3	0	0	1	0	1	2個真 literal
slack	0	0	0	2	2	各以 slack 2 補到4
target	1	1	1	4	4	

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

所有子集合都不能進位，才可反向解碼

若只對「依滿足指派選數」證明無進位，尚不足以證明反向：某個錯誤子集合也許靠進位偽造target。本構造對任意子集合都有變數欄≤2、clause欄≤6，故十進位下絕不進位。進位排除是所有候選的全稱命題。

每個數的位元長度O(n+k)，把十進位欄轉成binary可用逐欄乘10加digit的$O((n+k)^2)$簡單bit演算法；2n+2k個數也仍多項式。驗證子集合和時，至多加O(n+k)個O(n+k)位元整數，總和多出O(log(n+k))位元，亦多項式。

例如只取x1=false,x2=false,x3=false，兩clause的真literal數為0、2。第一欄即使兩個slack都取，也只有3，不能達4；不會向其他欄借出或借入一個位值來掩飾不滿足。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

編碼不變性、P 的封閉性與 coNP

兩種合理編碼若可相互在多項式時間轉換，P membership不變：先轉碼再求解，輸出長度受轉碼時間限制，組合仍多項式。Binary與unary整數編碼不具此雙向保證，因把b位元數2^(b−1)寫成unary需指數長輸出。

P對補集、聯集、交集封閉：把判定結果取反或依序執行兩個判定器即可。對串接L1L2，枚舉n+1個切點，每個執行兩個多項式判定器。對Kleene star L^*，用reachable[0]=true，對i<j以L判定x[i+1..j]是否可作下一段；$O(n^2)$次多項式判定與DP可決定reachable[n]，空字串直接接受。因此兩者也在P。

coNP={L：補集L在NP}，表示no instance有短證書，不等同「不是NP」。尚未知P=NP或NP=coNP。若NP-complete語言C也在coNP，任取A∈NP經A≤pC，則補集A≤p補集C；把補集C的NP證書接上多項式轉換，可知補集A∈NP，得到NP⊆coNP，再取補集得相等。此為條件推論，不能當成已證分離。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

NP-complete 證明鏈與各問題條件

教材使用的證明鏈可整理成：

`CIRCUIT-SAT <=_p SAT <=_p 3-CNF-SAT <=_p CLIQUE`。

再由 complement 得到 `CLIQUE <=_p INDEPENDENT-SET <=_p VERTEX-COVER` 的等價轉換。另由 `VERTEX-COVER <=_p HAMILTONIAN-CYCLE <=_p TSP-decision`。再以 3-CNF-SAT 到 SUBSET-SUM 的無進位十進位欄位構造，完成另一條 NP-complete 證明。

每個箭頭都要保留自己的 hypotheses：HC 用無向圖；TSP reduction 的 complete graph 可由原圖 polynomial-size 建出；subset-sum 的 t binary encoding 不能省；optimization TSP 與 decision TSP 不可混寫成同一語言。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

設計新 NP-complete 證明的檢查表

1. 把決策問題精確寫成 yes 集合，列出 instance 參數與 output。
2. 證明它 in NP：指定 certificate、長度上界、verifier、接受／拒絕條件。
3. 選一個已知 NP-complete source A，構造 `A <=_p f -> B`。
4. 證明 completeness direction：source yes 可構造 target yes。
5. 證明 soundness direction：target yes 可解回 source yes。
6. 證明 f 與 target verifier 都是 polynomial time，且 output size 是 polynomial。
7. 檢查退化 edge cases，例如空圖、k=0、重複 literals、圖不連通與 unreachable value。

若只有「看起來像」gadget，沒有第 4、5 步，就沒有約化證明。若只把新問題轉回已知 NP-complete，方向也反了。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 34。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study